

# PROJETO 2 - BANCO DE DADOS II

## Sistema de Gestão de Pedidos de Logística

**Autor:** bielbecke

**Data:** 21 de July de 2026

**Repositório:** [github.com/bielbecke/projeto2-bd2](https://github.com/bielbecke/projeto2-bd2)

## ÍNDICE

- 1. Visão Geral do Projeto
- 2. Arquitetura e Stack Tecnológico
- 3. Estrutura do Banco de Dados
- 4. Triggers e Regras de Negócio
- 5. Backend Flask (API REST)
- 6. Frontend React + Vite
- 7. Fluxo de Dados (Exemplo Prático)
- 8. Endpoints da API
- 9. Como Executar
- 10. Conclusão e Padrões Aplicados

## 1. Visão Geral do Projeto

**Objetivo Principal:** Desenvolver um sistema completo de gestão de pedidos de logística, integrando um banco de dados relacional (PostgreSQL) com uma API REST (Flask) e uma interface web moderna (React).

### Composição da Solução:

- ✓ **Banco de Dados:** PostgreSQL 12+ com triggers para validação de regras de negócio
- ✓ **Backend:** Python 3 + Flask para expor API REST
- ✓ **Frontend:** React 18 + Vite para interface interativa
- ✓ **Comunicação:** JSON via HTTP (REST)

### Principais Funcionalidades:

- Gestão completa de entidades: Cidades, Empresas, Clientes, Funcionários, Serviços e Pedidos
- Hierarquia de serviços com especialização em Guindastes e Transportes
- Cálculo automático e complexo de preços baseado em múltiplos fatores
- Validação de regras de negócio em tempo real via triggers SQL
- Dashboard com 6 relatórios analíticos (histogramas e rankings top-5)
- Atribuição dinâmica de funcionários a serviços

## 2. Arquitetura e Stack Tecnológico

| Camada       | Tecnologia                        | Função                                  |
|--------------|-----------------------------------|-----------------------------------------|
| Apresentação | React 18 + Vite + JavaScript ES6+ | Interface web interativa e responsiva   |
| Negócio      | Python 3 + Flask 3.0.3            | API REST para operações CRUD e cálculos |
| Persistência | PostgreSQL 12+                    | Armazenamento e validação via triggers  |
| Comunicação  | REST API + JSON + CORS            | Troca de dados entre cliente e servidor |

**Dependências do Backend:**

- Flask 3.0.3 - Framework web minimalista e extensível
- flask-cors 4.0.1 - Habilitação de CORS para requisições do frontend
- psycopg2-binary 2.9.9 - Driver PostgreSQL para Python

### 3. Estrutura do Banco de Dados

O banco de dados é composto por 13 tabelas principais, todas normalizadas e com relacionamentos bem definidos:

| Tabela       | Descrição                                 | Chave Primária                      |
|--------------|-------------------------------------------|-------------------------------------|
| idades       | Localidades onde empresas prestam serviço | nome_cidade, estado                 |
| empresas     | Empresas prestadoras de logística         | id_empresa                          |
| clientes     | Clientes que solicitam serviços           | cod_cliente                         |
| funcionarios | Funcionários das empresas                 | cpf_func                            |
| servicos     | Catálogo de serviços disponíveis          | nome_servico                        |
| guindastes   | Especialização: Serviços tipo Guindaste   | nome_servico (FK)                   |
| transportes  | Especialização: Serviços tipo Transporte  | nome_servico (FK)                   |
| oferece      | Relação empresa-cidade-serviço-preço      | id_empresa, cidade, estado, serviço |
| pedidos      | Solicitações dos clientes                 | codigo                              |
| solicitam    | Itens de serviço dentro de um pedido      | id_solicitacao                      |
| atendimento  | Alocação de funcionários aos serviços     | id_solicitacao, cpf_func            |
| vinculos     | Vínculo empresa-funcionário               | id_empresa, cpf_func                |

## 4. Triggers e Regras de Negócio

Os triggers implementados no PostgreSQL garantem a integridade das regras de negócio, executados automaticamente na camada de banco de dados:

| Regra                         | Descrição                                                                                                                          | Tipo                 |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| (a) Hierarquia DISJUNTA       | Um serviço NÃO pode ser Guindaste E Transporte simultaneamente. Hierarquia é parcial (pode não ser nenhum).                        | BEFORE INSERT        |
| (b) Cálculo de Preço por Item | $Preço = PreçoHora \times tempo\_duracao \times (1 + percentual/100)$ . Obediência a tabela de preços. Transporte usa faixa de car | BEFORE INSERT/UPDATE |
| (c) Preço Total do Pedido     | $Preço\_Total = SUM$ de todos os preços dos itens. Recalculado automaticamente e cadastrado.                                       | AFTER INSERT/UPDATE  |
| (d) Validação de Cidades      | Cidade de DESTINO e PARTIDA do pedido devem estar na base de dados da empresa.                                                     | BEFORE INSERT/UPDATE |

### Características da Implementação:

- Triggers BEFORE validam dados antes da inserção (regras a, d)
- Trigger de cálculo executa BEFORE para estar pronto quando o AFTER dispara
- Triggers AFTER somam totais e evitam inconsistências (regra c)
- Mensagens de erro são capturadas e retornadas como JSON via API
- Ordem de execução garantida: BEFORE (cálculo) → INSERT → AFTER (soma)

## 5. Backend Flask (API REST)

O backend é organizado em 7 blueprints (módulos de rotas), cada um responsável por uma entidade do domínio:

| Blueprint              | Entidade           | Endpoints Principais                                          |
|------------------------|--------------------|---------------------------------------------------------------|
| idades_blueprint       | Cidades            | GET /idades, POST /idades                                     |
| clientes_blueprint     | Clientes           | GET /clientes, POST /clientes                                 |
| empresas_blueprint     | Empresas           | GET /empresas, POST /empresas                                 |
| funcionarios_blueprint | Funcionários       | GET, POST /funcionarios; GET, POST, PATCH /vinculos           |
| servicos_blueprint     | Serviços & Ofertas | GET, POST /servicos; GET, POST /oferece                       |
| pedidos_blueprint      | Pedidos            | GET, POST /pedidos; PATCH resolver; POST atribuir funcionário |
| dashboard_blueprint    | Relatórios         | GET /dashboard/* (6 endpoints analíticos)                     |

**Padrão Arquitetural: MVC**

- **Model (database/\*.py):** Classes que encapsulam lógica de persistência e acesso a dados
- **View (rotas/\*.py):** Endpoints Flask que recebem requisições e retornam JSON
- **Controller (main.py):** Registro de blueprints e orquestração da aplicação

## 6. Frontend React + Vite

### Stack Tecnológico:

- React 18 - Biblioteca de componentes reativos
- Vite - Build tool e dev server de alta performance
- JavaScript ES6+ - Linguagem de programação
- CSS3 - Estilização com design tokens

### Estrutura de Arquivos:

- **src/api.js** - Camada HTTP com função fetch configurada
- **src/App.jsx** - Shell da aplicação e roteamento por abas
- **src/components/** - Componentes reutilizáveis (Sidebar, Cards, Gráficos, Toasts)
- **src/pages/** - 7 páginas principais (Dashboard, Pedidos, Empresas, Clientes, Serviços, Funcionários, Cidades)
- **src/styles.css** - Estilos globais com design tokens de cores e espaçamento

### Funcionalidades da Interface:

- ✓ Listagem de todas as entidades com agregação de dados relacionados
- ✓ Formulários de criação com validação de campos obrigatórios
- ✓ Modais para atribuição de funcionários a serviços
- ✓ Dashboard com 2 histogramas (quantidade e valor por cidade) + 4 rankings top-5
- ✓ Gestão de vínculos empresa-funcionário (criar e encerrar)
- ✓ Tratamento de erros com toasts (notificações visuais)
- ✓ Filtros dinâmicos (ex: funcionários por empresa)



## 7. Fluxo de Dados (Exemplo Prático)

### Cenário: Criação de um Pedido Completo com 2 Serviços

- 1■■■ **Usuário no Frontend:** Preenche formulário: empresa, cliente, cidades, itens de serviço com funcionários
- 2■■■ **Validação Frontend:** JavaScript verifica campos obrigatórios
- 3■■■ **Requisição HTTP:** POST para `http://localhost:8000/pedidos` com JSON no body
- 4■■■ **Backend recebe:** Flask desempacota JSON e chama `PedidosDatabase.criar_pedido()`
- 5■■■ **INSERT pedidos:** PostgreSQL insere pedido, trigger (d) valida se cidades existem na cobertura
- 6■■■ **INSERT solicitam:** Para cada item, insere serviço solicitado, trigger (c) calcula preço
- 7■■■ **INSERT atendimento:** Para cada funcionário de cada item, cria referência na tabela atendimento
- 8■■■ **AFTER trigger:** Recalcula `preco_total` em pedidos somando todos os itens
- 9■■■ **Resposta JSON:** Backend retorna `{codigo, preco_total}` com status 201 Created
- **Frontend exibe:** Toast de sucesso, atualiza lista de pedidos, limpa formulário

## 8. Endpoints da API

### ■ CIDADES

GET /cidades - Listar todas as cidades cadastradas

POST /cidades - Criar nova cidade (nome\_cidade, estado)

### ■ CLIENTES

GET /clientes - Listar todos os clientes

POST /clientes - Criar novo cliente

### ■ EMPRESAS

GET /empresas - Listar todas as empresas com telefones agregados

POST /empresas - Criar nova empresa (nome, endereco, telefones: [])

### ■■■ FUNCIONÁRIOS

GET /funcionarios - Listar todos ou filtrar por id\_empresa

GET /funcionarios?id\_empresa=1 - Funcionários de uma empresa

POST /funcionarios - Criar novo funcionário

GET /funcionarios/vinculos - Listar todos os vínculos empresa-funcionário

POST /funcionarios/vinculos - Criar novo vínculo

PATCH /funcionarios/vinculos/{id\_empresa}/{cpf\_func}/encerrar - Encerrar vínculo

### ■ SERVIÇOS & OFERTAS

GET /servicos - Listar todos os serviços (genéricos, Guindastes, Transportes)

POST /servicos - Criar serviço com tipo (generico, guindaste, transporte)

GET /oferece - Listar ofertas com filtros opcionais (id\_empresa, nome\_cidade, estado)

POST /oferece - Criar oferta (id\_empresa, nome\_cidade, estado, nome\_servico, preco\_hora)

### ■ PEDIDOS

GET /pedidos - Listar todos com itens agrupados e funcionários

POST /pedidos - Criar pedido com múltiplos itens e funcionários

PATCH /pedidos/{codigo}/resolver - Marcar pedido como resolvido (aceito + data\_resolucao)

POST /pedidos/servicos/{id\_solicitacao}/funcionarios - Atribuir funcionário a serviço

### ■ DASHBOARD (6 Relatórios Analíticos)

GET /dashboard/resumo - Totais: pedidos, faturado, pendentes, serviços

GET /dashboard/qtd-por-cidade - Quantidade de serviços por cidade (TODAS)

GET /dashboard/valor-por-cidade - Faturamento por cidade (TODAS)

GET /dashboard/top-cidades-valor - Top 5 cidades por faturamento

GET /dashboard/top-cidades-qtd - Top 5 cidades por quantidade de serviços

GET /dashboard/top-empresas-qtd - Top 5 empresas por quantidade de serviços

GET /dashboard/top-empresas-valor - Top 5 empresas por faturamento (aceitos + resolvidos)

## 9. Como Executar

### Pré-requisitos:

- Python 3.9+
- Node.js 16+ e npm
- PostgreSQL 12+
- Git

### Passo 1: Clonar o Repositório

```
git clone https://github.com/bielbecke/projeto2-bd2.git
cd projeto2-bd2
```

### Passo 2: Configurar PostgreSQL

1. Criar banco: createdb projeto\_bd2
2. Executar scripts SQL (em ordem):
  - backend/00\_schema.sql (tabelas)
  - backend/01\_dados.sql (dados iniciais)
  - backend/02\_triggers.sql (triggers e funções)
3. Configurar credenciais em backend/database/conector.py

### Passo 3: Backend (Terminal 1)

```
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python main.py
```

### Passo 4: Frontend (Terminal 2)

```
cd frontend
npm install
npm run dev
```

### Passo 5: Acessar

Frontend: <http://localhost:5173>

Backend: <http://localhost:8000>

Status: <http://localhost:8000/> → {"status": "ok"}

## 10. Conclusão e Padrões Aplicados

Este projeto demonstra uma implementação **production-ready** de um sistema de gestão de pedidos de logística, integrando corretamente as três camadas de uma arquitetura moderna:

### Camada de Dados (PostgreSQL):

Banco relacional normalizado com 13 tabelas bem estruturadas e triggers complexos que implementam regras de negócio sofisticadas (cálculo de preços, validações, integridade referencial). Todos os cálculos e validações críticas ocorrem no banco, garantindo que não podem ser burlados pelo frontend ou API.

### Camada de Negócio (Flask):

API REST que expõe funcionalidades de forma segura através de 7 blueprints organizados. Traduz erros de negócio em mensagens amigáveis ao usuário e orquestra a lógica transacional com precisão.

### Camada de Apresentação (React):

Interface moderna, responsiva e intuitiva para gestão de entidades e visualização de dados analíticos. Mantém estado local, comunica com backend via HTTP e oferece feedback visual em tempo real.

### Padrões Arquiteturais Aplicados:

- ✓ **MVC** - Separação clara entre Model (banco), View (frontend) e Controller (rotas)
- ✓ **REST API** - Endpoints semânticos com métodos HTTP apropriados
- ✓ **Repository Pattern** - Database classes encapsulam lógica de acesso a dados
- ✓ **Trigger-based Validation** - Regras críticas implementadas no banco
- ✓ **Component-based UI** - React com componentes reutilizáveis
- ✓ **Error Handling** - Exceções traduzidas em JSON com contexto útil
- ✓ **CORS** - Comunicação segura entre domínios diferentes

O sistema está pronto para evolução: novos endpoints, relatórios e funcionalidades podem ser adicionados mantendo a coesão arquitetural. As regras de negócio estão centralizadas no banco de dados, facilitando manutenção e garantindo consistência em qualquer ponto de acesso aos dados.

Relatório gerado em 21/07/2026 às 09:07

Projeto 2 - Banco de Dados II | [github.com/bielbecke/projeto2-bd2](https://github.com/bielbecke/projeto2-bd2)